COMMANDS_REFERENCE — SICL CLI

Estado: DOCUMENTACIÓN CANÓNICA DE SINTAXIS **Fuente de verdad:** `src/sicl/cli.py` en `main`. **Baseline:** `246417391dfb85c63596e1650faef39466378954`.

Esta referencia describe el dispatcher real. Si contradice `USER_MANUAL.md`, `OPERATIONS_MANUAL.md` u otro documento, prevalece esta referencia. Los comandos marcados **HTTP-only** no son aceptados por el CLI local.

1. Cómo usar esta referencia

Cada ficha contiene nombre, sintaxis exacta, argumentos, ejemplo, comportamiento y errores frecuentes. Los valores entre `< >` son obligatorios; los valores entre `[ ]` son opcionales. Use comillas para argumentos con espacios y JSON inline con comillas simples externas en shells POSIX.

1. PROYECTO

/PROJECT CREATE

Sintaxis: `/PROJECT CREATE <project_id> <name> [spatial_scope] [temporal_scope]`

Ejemplo:

```
/PROJECT CREATE UPAO-001 "UPAO Plaza" parcela_sitio proyecto
```

Validación y comportamiento: Requiere project_id y name; scopes se validan contra los enums del dominio; crea Project y evento PROJECT_CREATED.

Errores comunes: `PROJECT_ALREADY_EXISTS, INVALID_ARGUMENT, INVALID_SCOPE`

/PROJECT OPEN

Sintaxis: `/PROJECT OPEN <project_id>`

Ejemplo:

```
/PROJECT OPEN UPAO-001
```

Validación y comportamiento: Requiere un proyecto existente; lo convierte en el proyecto actual.

Errores comunes: `PROJECT_NOT_FOUND, INVALID_ARGUMENT`

/PROJECT SET SCOPE

Sintaxis: `/PROJECT SET SCOPE <spatial_scope> [temporal_scope]`

Ejemplo:

```
/PROJECT SET SCOPE edificacion proyecto
```

Validación y comportamiento: Requiere proyecto abierto; actualiza scopes y versión.

Errores comunes: `PROJECT_NOT_FOUND, INVALID_SCOPE, INVALID_ARGUMENT`

/PROJECT IMPORT OBJECTIVE

Sintaxis: `/PROJECT IMPORT OBJECTIVE <source_project_id> <objective_id>`

Ejemplo:

```
/PROJECT IMPORT OBJECTIVE CITY-001 OBJ-123
```

Validación y comportamiento: Requiere proyecto abierto, relación multiescala válida y objetivo existente.

Errores comunes: `PROJECT_NOT_FOUND, OBJECTIVE_NOT_FOUND, SCALE_RELATION_REQUIRED`

/PROJECT SHOW

Sintaxis: `/PROJECT SHOW`

Ejemplo:

```
/PROJECT SHOW
```

Validación y comportamiento: Devuelve el snapshot del proyecto actual.

Errores comunes: `PROJECT_NOT_FOUND`

/PROJECT LIST

Sintaxis: `/PROJECT LIST`

Ejemplo:

```
/PROJECT LIST
```

Validación y comportamiento: Lista proyectos persistidos.

Errores comunes: `INVALID_STATE`

2. ESTADO Y AUDITORÍA

/STAGE SET

Sintaxis: `/STAGE SET <DRAFT|ACTIVE|CLOSED>`

Ejemplo:

```
/STAGE SET ACTIVE
```

Validación y comportamiento: Requiere proyecto abierto; solo acepta DRAFT, ACTIVE o CLOSED.

Errores comunes: `INVALID_ARGUMENT, PROJECT_NOT_FOUND`

/STATUS

Sintaxis: `/STATUS`

Ejemplo:

```
/STATUS
```

Validación y comportamiento: Devuelve estado y snapshot del proyecto actual.

Errores comunes: `PROJECT_NOT_FOUND`

/HISTORY

Sintaxis: `/HISTORY`

Ejemplo:

```
/HISTORY
```

Validación y comportamiento: Devuelve eventos del proyecto en orden temporal.

Errores comunes: `PROJECT_NOT_FOUND`

3. CONOCIMIENTO

/FACT SET

Sintaxis: `/FACT SET <statement> <source>`

Ejemplo:

```
/FACT SET "El lote tiene 2000 m2" CATASTRO
```

Validación y comportamiento: Requiere statement y source; crea Fact y evento append-only.

Errores comunes: `INVALID_ARGUMENT, PROJECT_NOT_FOUND`

/ASSUMPTION SET

Sintaxis: `/ASSUMPTION SET <statement> <basis>`

Ejemplo:

```
/ASSUMPTION SET "La ocupación será 80 por ciento" ESTIMACION
```

Validación y comportamiento: Requiere statement y basis; crea Assumption separada de Fact.

Errores comunes: `INVALID_ARGUMENT, PROJECT_NOT_FOUND`

/EVIDENCE ADD

Sintaxis: `/EVIDENCE ADD <evidence_id> <statement> <type> [source_id] [url]`

Ejemplo:

```
/EVIDENCE ADD EVD-001 "Frente de 40 m" OBSERVATION
```

Validación y comportamiento: Tipo: DOCUMENT, OBSERVATION, MEASUREMENT, REFERENCE, TESTIMONY, NORMATIVE u OTHER.

Errores comunes: `EVIDENCE_ALREADY_EXISTS, INVALID_EVIDENCE_TYPE, INVALID_ARGUMENT`

/EVIDENCE LIST

Sintaxis: `/EVIDENCE LIST`

Ejemplo:

```
/EVIDENCE LIST
```

Validación y comportamiento: Lista evidencia del proyecto actual.

Errores comunes: `PROJECT_NOT_FOUND`

/EVIDENCE SHOW

Sintaxis: `/EVIDENCE SHOW <evidence_id>`

Ejemplo:

```
/EVIDENCE SHOW EVD-001
```

Validación y comportamiento: Requiere evidencia existente.

Errores comunes: `EVIDENCE_NOT_FOUND, INVALID_ARGUMENT`

4. FUENTES — HTTP-only

/SOURCE ADD

Sintaxis: `HTTP: POST /v1/projects/{project_id}/sources`

Ejemplo:

```
POST /v1/projects/UPAO-001/sources
```

Validación y comportamiento: No es comando CLI. Crear Source requiere API HTTP v1.

Errores comunes: `No aplica en CLI`

/SOURCE LIST

Sintaxis: `HTTP: GET /v1/projects/{project_id}/sources`

Ejemplo:

```
GET /v1/projects/UPAO-001/sources
```

Validación y comportamiento: No es comando CLI.

Errores comunes: `No aplica en CLI`

/SOURCE SHOW

Sintaxis: `HTTP: GET /v1/projects/{project_id}/sources/{source_id}`

Ejemplo:

```
GET /v1/projects/UPAO-001/sources/CATASTRO
```

Validación y comportamiento: No es comando CLI.

Errores comunes: `No aplica en CLI`

5. DISEÑO

/OBJECTIVE SET

Sintaxis: `/OBJECTIVE SET <key> <MAXIMIZE|MINIMIZE> <value>`

Ejemplo:

```
/OBJECTIVE SET CLIMATE MAXIMIZE 80
```

Validación y comportamiento: Direction obligatoria; crea Objective.

Errores comunes: `OBJECTIVE_DIRECTION_REQUIRED, INVALID_ARGUMENT, PROJECT_NOT_FOUND`

/CONSTRAINT SET

Sintaxis: `/CONSTRAINT SET <key> <operator> <value> [unit]`

Ejemplo:

```
/CONSTRAINT SET HEIGHT ⩽ 6 FLOORS
```

Validación y comportamiento: Constraint MVP es siempre hard=true.

Errores comunes: `INVALID_ARGUMENT, PROJECT_NOT_FOUND`

/ROLE ADD

Sintaxis: `/ROLE ADD <name> [actor]`

Ejemplo:

```
/ROLE ADD ARCHITECT "Yvan Castillo"
```

Validación y comportamiento: Crea Role asociado al proyecto.

Errores comunes: `INVALID_ARGUMENT, PROJECT_NOT_FOUND`

6. ALTERNATIVAS

/ALTERNATIVE CREATE

Sintaxis: `/ALTERNATIVE CREATE <name>`

Ejemplo:

```
/ALTERNATIVE CREATE CONVENCIONAL_A
```

Validación y comportamiento: Acepta exactamente un name; description no es argumento CLI.

Errores comunes: `INVALID_ARGUMENT, CONFLICT, PROJECT_NOT_FOUND`

/ALTERNATIVE SET

Sintaxis: `/ALTERNATIVE SET <name> <parameter> <value>`

Ejemplo:

```
/ALTERNATIVE SET CONVENCIONAL_A FLOORS 6
```

Validación y comportamiento: Requiere alternativa existente; agrega o actualiza parámetro.

Errores comunes: `CANDIDATE_NOT_FOUND, INVALID_ARGUMENT`

/ALTERNATIVE LIST

Sintaxis: `/ALTERNATIVE LIST`

Ejemplo:

```
/ALTERNATIVE LIST
```

Validación y comportamiento: Lista alternativas del proyecto.

Errores comunes: `PROJECT_NOT_FOUND`

/ALTERNATIVE PROMOTE

Sintaxis: `/ALTERNATIVE PROMOTE <alternative_id>`

Ejemplo:

```
/ALTERNATIVE PROMOTE ALT-123
```

Validación y comportamiento: Promueve GeneratedAlternative; requiere candidato existente.

Errores comunes: `CANDIDATE_NOT_FOUND, INVALID_STATE`

7. EVALUACIÓN

/EVALUATE

Sintaxis: `/EVALUATE <alternative> <objective> <value> <unit> <confidence> <source>`

Ejemplo:

```
/EVALUATE CONVENCIONAL_A CLIMATE 80 SCORE 0.9 USER_INPUT
```

Validación y comportamiento: Requiere alternativa y objetivo; confidence debe ser numérica.

Errores comunes: `OBJECTIVE_NOT_FOUND, CANDIDATE_NOT_FOUND, INVALID_ARGUMENT`

/COMPARE

Sintaxis: `/COMPARE <alternative_1> <alternative_2>`

Ejemplo:

```
/COMPARE CONVENCIONAL_A BIOCLIMATICA_B
```

Validación y comportamiento: Compara alternativas y evaluaciones existentes.

Errores comunes: `CANDIDATE_NOT_FOUND, INVALID_STATE`

8. GOBERNANZA

/HUMAN REVIEW

Sintaxis: `/HUMAN REVIEW <actor> <timestamp_utc> <review> <reason> [authority]`

Ejemplo:

```
/HUMAN REVIEW architect-yvan 2026-09-16T07:00:00Z "Revisé la recomendació
```

Validación y comportamiento: Timestamp UTC obligatorio; crea HumanReview APPROVED.

Errores comunes: `HUMAN_AUTHORITY_REQUIRED, INVALID_ARGUMENT, PROJECT_NOT_FOUND`

/DECISION RECORD

Sintaxis: `/DECISION RECORD <statement> <actor> <authority>`

Ejemplo:

```
/DECISION RECORD "Adoptar CONVENCIONAL_A" architect-yvan Architect
```

Validación y comportamiento: Requiere HumanReview APPROVED previo y autoridad humana; nunca es creado por Recommendation.

Errores comunes: `HUMAN_REVIEW_REQUIRED, HUMAN_AUTHORITY_REQUIRED, DECISIONAL_ACTOR_REQUIRED`

/COUNCIL

Sintaxis: `HTTP-only: GET /v1/projects/{project_id}/council`

Ejemplo:

```
GET /v1/projects/UPAO-001/council
```

Validación y comportamiento: No es comando CLI local; lectura HTTP.

Errores comunes: `No aplica en CLI`

9. SIMULACIÓN

/SIMULATE RUN

Sintaxis: `/SIMULATE RUN <simulation_type> <method> [json_params|key=value ...]`

Ejemplo:

```
/SIMULATE RUN DETERMINISTIC deterministic_basic_v1 '{"alternative_id":"AL
```

Validación y comportamiento: Simulation type y method deben coincidir; params pueden ser JSON o key=value.

Errores comunes: `METHOD_NOT_FOUND, METHOD_TYPE_MISMATCH, INVALID_INPUTS, PARAMETER_NOT_FOUND`

/SIMULATE MONTE_CARLO

Sintaxis: `/SIMULATE MONTE_CARLO <alternative_id> <objective_id> <parameter_name> <distribution_json> [iterations]`

Ejemplo:

```
/SIMULATE MONTE_CARLO ALT-1 OBJ-1 AREA '{"type":"NORMAL","parameters":{"m
```

Validación y comportamiento: Distribuciones NORMAL, UNIFORM o TRIANGULAR; iterations 1..10000; seed fija 20260916.

Errores comunes: `INVALID_DISTRIBUTION, INVALID_INPUTS, PARAMETER_NOT_FOUND`

/SIMULATE METHODS

Sintaxis: `/SIMULATE METHODS`

Ejemplo:

```
/SIMULATE METHODS
```

Validación y comportamiento: Lista métodos registrados.

Errores comunes: `INVALID_STATE`

/SIMULATE LIST

Sintaxis: `/SIMULATE LIST`

Ejemplo:

```
/SIMULATE LIST
```

Validación y comportamiento: Lista simulaciones del proyecto.

Errores comunes: `PROJECT_NOT_FOUND`

/SIMULATE SHOW

Sintaxis: `/SIMULATE SHOW <simulation_id>`

Ejemplo:

```
/SIMULATE SHOW SIM-123
```

Validación y comportamiento: Requiere simulación existente.

Errores comunes: `SIMULATION_NOT_FOUND, INVALID_ARGUMENT`

10. MULTIOBJETIVO

/MULTIOBJECTIVE PARETO

Sintaxis: `/MULTIOBJECTIVE PARETO <objective_key_1> <objective_key_2> [...]`

Ejemplo:

```
/MULTIOBJECTIVE PARETO ENERGY_SAVINGS CONSTRUCTION_COST
```

Validación y comportamiento: Requiere al menos dos objetivos existentes con dirección.

Errores comunes: `OBJECTIVE_NOT_FOUND, INVALID_ARGUMENT, INVALID_STATE`

/MULTIOBJECTIVE TRADEOFFS

Sintaxis: `/MULTIOBJECTIVE TRADEOFFS <objective_key_1> <objective_key_2>`

Ejemplo:

```
/MULTIOBJECTIVE TRADEOFFS ENERGY_SAVINGS CONSTRUCTION_COST
```

Validación y comportamiento: Requiere exactamente dos objetivos.

Errores comunes: `OBJECTIVE_NOT_FOUND, INVALID_ARGUMENT`

/MULTIOBJECTIVE LIST

Sintaxis: `/MULTIOBJECTIVE LIST`

Ejemplo:

```
/MULTIOBJECTIVE LIST
```

Validación y comportamiento: Lista resultados multiobjetivo.

Errores comunes: `PROJECT_NOT_FOUND`

/MULTIOBJECTIVE SHOW

Sintaxis: `/MULTIOBJECTIVE SHOW <multiobjective_id>`

Ejemplo:

```
/MULTIOBJECTIVE SHOW MO-123
```

Validación y comportamiento: Requiere resultado existente.

Errores comunes: `MULTIOBJECTIVE_NOT_FOUND, INVALID_ARGUMENT`

11. GENERACIÓN

/GENERATE DESIGN

Sintaxis: `/GENERATE DESIGN <method> <parameters_json>`

Ejemplo:

```
/GENERATE DESIGN parametric_grid_v1 '{"FLOORS":[4,6]}'
```

Validación y comportamiento: Genera candidatos sin decidir; métodos dependen del catálogo.

Errores comunes: `METHOD_NOT_FOUND, INVALID_INPUTS`

/GENERATE METHODS

Sintaxis: `/GENERATE METHODS`

Ejemplo:

```
/GENERATE METHODS
```

Validación y comportamiento: Lista métodos de generación.

Errores comunes: `INVALID_STATE`

/GENERATE LIST

Sintaxis: `/GENERATE LIST`

Ejemplo:

```
/GENERATE LIST
```

Validación y comportamiento: Lista GeneratedAlternative.

Errores comunes: `PROJECT_NOT_FOUND`

/GENERATE SHOW

Sintaxis: `/GENERATE SHOW <generation_id>`

Ejemplo:

```
/GENERATE SHOW GEN-123
```

Validación y comportamiento: Requiere generación existente.

Errores comunes: `GENERATION_NOT_FOUND, INVALID_ARGUMENT`

/GENERATE

Sintaxis: `/GENERATE <method> <objective_keys_csv>`

Ejemplo:

```
/GENERATE evolutionary_v1 ENERGY_SAVINGS,CONSTRUCTION_COST
```

Validación y comportamiento: Genera alternativas; no crea Decision.

Errores comunes: `METHOD_NOT_FOUND, INVALID_INPUTS, LLM_NOT_CONFIGURED`

/PARETO

Sintaxis: `/PARETO <objective_1> <objective_2>`

Ejemplo:

```
/PARETO ENERGY_SAVINGS CONSTRUCTION_COST
```

Validación y comportamiento: Atajo de análisis Pareto.

Errores comunes: `OBJECTIVE_NOT_FOUND, INVALID_ARGUMENT`

/TRADEOFF_MATRIX

Sintaxis: `/TRADEOFF_MATRIX <objective_1> <objective_2>`

Ejemplo:

```
/TRADEOFF_MATRIX ENERGY_SAVINGS CONSTRUCTION_COST
```

Validación y comportamiento: Atajo de matriz de trade-offs.

Errores comunes: `OBJECTIVE_NOT_FOUND, INVALID_ARGUMENT`

12. PRINCIPIOS DE DISEÑO

/DESIGN PRINCIPLES

Sintaxis: `/DESIGN PRINCIPLES`

Ejemplo:

```
/DESIGN PRINCIPLES
```

Validación y comportamiento: Lista principios de diseño disponibles.

Errores comunes: `INVALID_STATE`

/DESIGN PRINCIPLE

Sintaxis: `/DESIGN PRINCIPLE <principle_id>`

Ejemplo:

```
/DESIGN PRINCIPLE BIOCLIMATIC
```

Validación y comportamiento: Muestra un principio registrado.

Errores comunes: `PRINCIPLE_NOT_FOUND, INVALID_ARGUMENT`

13. PLANIFICACIÓN

/PLANNING ADD

Sintaxis: `/PLANNING ADD <id> <type> <name> [jurisdiction] [url]`

Ejemplo:

```
/PLANNING ADD PI-001 ZONING "Plan urbano" Trujillo https://example.org
```

Validación y comportamiento: Type debe ser PlanningInstrumentType válido.

Errores comunes: `CONFLICT, INVALID_ARGUMENT`

/PLANNING LIST

Sintaxis: `/PLANNING LIST`

Ejemplo:

```
/PLANNING LIST
```

Validación y comportamiento: Lista instrumentos.

Errores comunes: `INVALID_STATE`

/PLANNING SHOW

Sintaxis: `/PLANNING SHOW <instrument_id>`

Ejemplo:

```
/PLANNING SHOW PI-001
```

Validación y comportamiento: Muestra instrumento existente.

Errores comunes: `PLANNING_INSTRUMENT_NOT_FOUND`

/PLANNING TYPES

Sintaxis: `/PLANNING TYPES`

Ejemplo:

```
/PLANNING TYPES
```

Validación y comportamiento: Lista tipos válidos.

Errores comunes: `INVALID_STATE`

14. REGULACIÓN

/REGULATION ADD

Sintaxis: `/REGULATION ADD <id> <code> <title> [jurisdiction] [source_url]`

Ejemplo:

```
/REGULATION ADD REG-001 RNE "Reglamento Nacional" Peru https://example.or
```

Validación y comportamiento: Registra regulación con estado NO_VERIFICADA.

Errores comunes: `CONFLICT, INVALID_ARGUMENT`

/REGULATION LIST

Sintaxis: `/REGULATION LIST`

Ejemplo:

```
/REGULATION LIST
```

Validación y comportamiento: Lista regulaciones.

Errores comunes: `INVALID_STATE`

/REGULATION SHOW

Sintaxis: `/REGULATION SHOW <regulation_id>`

Ejemplo:

```
/REGULATION SHOW REG-001
```

Validación y comportamiento: Muestra regulación.

Errores comunes: `REGULATION_NOT_FOUND`

/REGULATION STATUS

Sintaxis: `/REGULATION STATUS <regulation_id> <status>`

Ejemplo:

```
/REGULATION STATUS REG-001 VERIFICADA
```

Validación y comportamiento: Actualiza estado de regulación.

Errores comunes: `REGULATION_NOT_FOUND, INVALID_ARGUMENT`

/INTERPRET ADD

Sintaxis: `/INTERPRET ADD <id> <regulation_id> <article> <text>`

Ejemplo:

```
/INTERPRET ADD INT-001 REG-001 Art-10 "Interpretación técnica"
```

Validación y comportamiento: Regulación debe existir; incluye disclaimer legal.

Errores comunes: `REGULATION_NOT_FOUND, CONFLICT, INVALID_ARGUMENT`

/INTERPRET LIST

Sintaxis: `/INTERPRET LIST [regulation_id]`

Ejemplo:

```
/INTERPRET LIST REG-001
```

Validación y comportamiento: Lista interpretaciones opcionalmente filtradas.

Errores comunes: `INVALID_ARGUMENT`

/INTERPRET REVIEW

Sintaxis: `/INTERPRET REVIEW <interpretation_id> <actor> <authority>`

Ejemplo:

```
/INTERPRET REVIEW INT-001 architect-yvan Architect
```

Validación y comportamiento: Requiere actor y authority no vacíos.

Errores comunes: `INTERPRETATION_NOT_FOUND, INVALID_ARGUMENT`

/SNAPSHOT CREATE

Sintaxis: `/SNAPSHOT CREATE <snapshot_id> <project_id> <cut_date>`

Ejemplo:

```
/SNAPSHOT CREATE SNAP-001 UPAO-001 2026-09-16
```

Validación y comportamiento: cut_date debe ser ISO date.

Errores comunes: `PROJECT_NOT_FOUND, INVALID_ARGUMENT`

/SNAPSHOT LIST

Sintaxis: `/SNAPSHOT LIST`

Ejemplo:

```
/SNAPSHOT LIST
```

Validación y comportamiento: Lista snapshots.

Errores comunes: `INVALID_STATE`

/SNAPSHOT FREEZE

Sintaxis: `/SNAPSHOT FREEZE <snapshot_id>`

Ejemplo:

```
/SNAPSHOT FREEZE SNAP-001
```

Validación y comportamiento: Congela snapshot existente.

Errores comunes: `SNAPSHOT_NOT_FOUND, INVALID_ARGUMENT`

15. MULTIESCALA

/SCALE PARENT

Sintaxis: `/SCALE PARENT <project_id>`

Ejemplo:

```
/SCALE PARENT CITY-001
```

Validación y comportamiento: Busca proyecto padre en relaciones.

Errores comunes: `PROJECT_NOT_FOUND`

/SCALE CHILDREN

Sintaxis: `/SCALE CHILDREN <project_id>`

Ejemplo:

```
/SCALE CHILDREN CITY-001
```

Validación y comportamiento: Lista hijos relacionados.

Errores comunes: `PROJECT_NOT_FOUND`

/SCALE RELATE

Sintaxis: `/SCALE RELATE <parent_id> <child_id> <relation_type>`

Ejemplo:

```
/SCALE RELATE CITY-001 SITE-001 CONTAINS
```

Validación y comportamiento: Proyectos deben existir; relation_type se valida.

Errores comunes: `PROJECT_NOT_FOUND, INVALID_SCOPE_RELATION`

/SCALE RELATIONS

Sintaxis: `/SCALE RELATIONS <project_id>`

Ejemplo:

```
/SCALE RELATIONS CITY-001
```

Validación y comportamiento: Lista relaciones del proyecto.

Errores comunes: `PROJECT_NOT_FOUND`

16. ACTORES

/ACTOR ADD

Sintaxis: `/ACTOR ADD <id> <role> <name> <authority> [interests_json constraints_json]`

Ejemplo:

```
/ACTOR ADD architect-yvan ARCHITECT "Yvan" DECISIONAL
```

Validación y comportamiento: Role y authority se validan; JSON opcional debe ser lista.

Errores comunes: `CONFLICT, INVALID_ARGUMENT`

/ACTOR LIST

Sintaxis: `/ACTOR LIST`

Ejemplo:

```
/ACTOR LIST
```

Validación y comportamiento: Lista actores.

Errores comunes: `PROJECT_NOT_FOUND`

/ACTOR SHOW

Sintaxis: `/ACTOR SHOW <actor_id>`

Ejemplo:

```
/ACTOR SHOW architect-yvan
```

Validación y comportamiento: Muestra actor.

Errores comunes: `ACTOR_NOT_FOUND`

/POSITION ADD

Sintaxis: `/POSITION ADD <actor_id> <subject_type> <subject_id> <stance> <statement>`

Ejemplo:

```
/POSITION ADD architect-yvan ALTERNATIVE ALT-1 SUPPORT "Apoyo"
```

Validación y comportamiento: Actor, subject type y stance se validan.

Errores comunes: `ACTOR_NOT_FOUND, INVALID_ARGUMENT`

/POSITION LIST

Sintaxis: `/POSITION LIST [actor_id] [subject_id]`

Ejemplo:

```
/POSITION LIST architect-yvan
```

Validación y comportamiento: Lista posiciones con filtros opcionales.

Errores comunes: `ACTOR_NOT_FOUND, INVALID_ARGUMENT`

17. TIEMPO

/CYCLE CREATE

Sintaxis: `/CYCLE CREATE <cycle_id> <horizon> <start_date> <end_date>`

Ejemplo:

```
/CYCLE CREATE CYCLE-2030 2030 2026-01-01 2030-12-31
```

Validación y comportamiento: Horizon y fechas se validan.

Errores comunes: `INVALID_ARGUMENT, CONFLICT`

/CYCLE LIST

Sintaxis: `/CYCLE LIST`

Ejemplo:

```
/CYCLE LIST
```

Validación y comportamiento: Lista ciclos.

Errores comunes: `PROJECT_NOT_FOUND`

/CYCLE SHOW

Sintaxis: `/CYCLE SHOW <cycle_id>`

Ejemplo:

```
/CYCLE SHOW CYCLE-2030
```

Validación y comportamiento: Muestra ciclo.

Errores comunes: `CYCLE_NOT_FOUND`

/SCENARIO CREATE

Sintaxis: `/SCENARIO CREATE <branch_id> <cycle_id> <name>`

Ejemplo:

```
/SCENARIO CREATE BASE-2030 CYCLE-2030 "Base"
```

Validación y comportamiento: Ciclo debe existir.

Errores comunes: `CYCLE_NOT_FOUND, CONFLICT`

/SCENARIO LIST

Sintaxis: `/SCENARIO LIST`

Ejemplo:

```
/SCENARIO LIST
```

Validación y comportamiento: Lista escenarios.

Errores comunes: `PROJECT_NOT_FOUND`

/SCENARIO SELECT

Sintaxis: `/SCENARIO SELECT <branch_id>`

Ejemplo:

```
/SCENARIO SELECT BASE-2030
```

Validación y comportamiento: Selecciona rama de escenario.

Errores comunes: `SCENARIO_NOT_FOUND`

/EVOLUTION CREATE

Sintaxis: `/EVOLUTION CREATE <evolution_id> <branch_id> <version> <title>`

Ejemplo:

```
/EVOLUTION CREATE EVO-001 BASE-2030 1 "Cambio de densidad"
```

Validación y comportamiento: Crea evolución append-only.

Errores comunes: `SCENARIO_NOT_FOUND, CONFLICT`

/EVOLUTION ADD

Sintaxis: `/EVOLUTION ADD <evolution_id> <key> <value>`

Ejemplo:

```
/EVOLUTION ADD EVO-001 FAR 5
```

Validación y comportamiento: Agrega cambio a evolución.

Errores comunes: `EVOLUTION_NOT_FOUND, INVALID_ARGUMENT`

/EVOLUTION APPLY

Sintaxis: `/EVOLUTION APPLY <evolution_id>`

Ejemplo:

```
/EVOLUTION APPLY EVO-001
```

Validación y comportamiento: Aplica evolución autorizada al escenario.

Errores comunes: `EVOLUTION_NOT_FOUND, INVALID_STATE`

/EVOLUTION LIST

Sintaxis: `/EVOLUTION LIST [branch_id]`

Ejemplo:

```
/EVOLUTION LIST BASE-2030
```

Validación y comportamiento: Lista evoluciones.

Errores comunes: `INVALID_ARGUMENT`

/EVOLUTION SHOW

Sintaxis: `/EVOLUTION SHOW <evolution_id>`

Ejemplo:

```
/EVOLUTION SHOW EVO-001
```

Validación y comportamiento: Muestra evolución.

Errores comunes: `EVOLUTION_NOT_FOUND`

18. MEMORIA INSTITUCIONAL

/MEMORY LIST

Sintaxis: `/MEMORY LIST`

Ejemplo:

```
/MEMORY LIST
```

Validación y comportamiento: Lista memorias activas.

Errores comunes: `PROJECT_NOT_FOUND`

/MEMORY SHOW

Sintaxis: `/MEMORY SHOW <memory_id>`

Ejemplo:

```
/MEMORY SHOW MEM-001
```

Validación y comportamiento: Muestra memoria.

Errores comunes: `MEMORY_NOT_FOUND`

/MEMORY EXTRACT

Sintaxis: `/MEMORY EXTRACT <type> <title> <content>`

Ejemplo:

```
/MEMORY EXTRACT LESSON_LEARNED "Lección" "Contenido"
```

Validación y comportamiento: Crea memoria desde conocimiento del proyecto.

Errores comunes: `INVALID_ARGUMENT`

/MEMORY REVOKE

Sintaxis: `/MEMORY REVOKE <memory_id>`

Ejemplo:

```
/MEMORY REVOKE MEM-001
```

Validación y comportamiento: Revoca memoria sin borrarla.

Errores comunes: `MEMORY_NOT_FOUND, MEMORY_REVOKED`

/MEMORY APPLY

Sintaxis: `/MEMORY APPLY <memory_id>`

Ejemplo:

```
/MEMORY APPLY MEM-001
```

Validación y comportamiento: Aplica memoria vigente.

Errores comunes: `MEMORY_NOT_FOUND, MEMORY_REVOKED`

/MEMORY TYPES

Sintaxis: `/MEMORY TYPES`

Ejemplo:

```
/MEMORY TYPES
```

Validación y comportamiento: Lista tipos de memoria.

Errores comunes: `INVALID_STATE`

19. EXPORTACIÓN

/REPORT

Sintaxis: `No registrado en CLI actual`

Ejemplo:

```
Usar /EXPORT REPORT <path>
```

Validación y comportamiento: La salida report se genera mediante export_report_command.

Errores comunes: `UNKNOWN_COMMAND`

/EXPORT PROJECT

Sintaxis: `/EXPORT PROJECT [path]`

Ejemplo:

```
/EXPORT PROJECT exports/UPAO-001.json
```

Validación y comportamiento: Exporta snapshot JSON.

Errores comunes: `PROJECT_NOT_FOUND, INVALID_ARGUMENT`

/EXPORT CSV

Sintaxis: `/EXPORT CSV [directory]`

Ejemplo:

```
/EXPORT CSV exports
```

Validación y comportamiento: Exporta tablas CSV.

Errores comunes: `PROJECT_NOT_FOUND, IMPORT_FAILED`

/EXPORT REPORT

Sintaxis: `/EXPORT REPORT [path]`

Ejemplo:

```
/EXPORT REPORT exports/report.json
```

Validación y comportamiento: Exporta reporte derivado.

Errores comunes: `PROJECT_NOT_FOUND, INVALID_ARGUMENT`

/IMPORT CSV

Sintaxis: `/IMPORT CSV <directory>`

Ejemplo:

```
/IMPORT CSV imports/UPAO-001
```

Validación y comportamiento: Importa CSV al proyecto actual.

Errores comunes: `IMPORT_FAILED, INVALID_ARGUMENT`

20. OTROS

/HELP

Sintaxis: `/HELP`

Ejemplo:

```
/HELP
```

Validación y comportamiento: Lista el catálogo de comandos expuesto por el CLI.

Errores comunes: `INVALID_STATE`

/TRADEOFFS

Sintaxis: `No es un comando independiente registrado`

Ejemplo:

```
Usar /MULTIOBJECTIVE TRADEOFFS A B
```

Validación y comportamiento: El dispatcher usa /TRADEOFF_MATRIX y /MULTIOBJECTIVE TRADEOFFS.

Errores comunes: `UNKNOWN_COMMAND`

/DASHBOARD

Sintaxis: `No es un comando independiente registrado`

Ejemplo:

```
Usar /STATUS o /PROJECT SHOW
```

Validación y comportamiento: No existe en el dispatcher actual.

Errores comunes: `UNKNOWN_COMMAND`

/SITE INTELLIGENCE

Sintaxis: `/SITE INTELLIGENCE <location>`

Ejemplo:

```
/SITE INTELLIGENCE "Trujillo, Peru"
```

Validación y comportamiento: Consulta inteligencia de sitio y registra observaciones.

Errores comunes: `INVALID_ARGUMENT, INVALID_STATE`

/AGENT RUN

Sintaxis: `/AGENT RUN <agent_type> <alternative_id>`

Ejemplo:

```
/AGENT RUN BIOCLIMATIC CONVENCIONAL_A
```

Validación y comportamiento: Ejecuta agente experto; genera evaluaciones, no decisiones.

Errores comunes: `CANDIDATE_NOT_FOUND, INVALID_ARGUMENT`

/DEBATE

Sintaxis: `/DEBATE <alternative_id>`

Ejemplo:

```
/DEBATE CONVENCIONAL_A
```

Validación y comportamiento: Read-only; resume evaluaciones de agentes.

Errores comunes: `CANDIDATE_NOT_FOUND, INVALID_ARGUMENT`

/DECISION RECORD

Sintaxis: `/DECISION RECORD <statement> <actor> <authority>`

Ejemplo:

```
/DECISION RECORD "Adoptar A" architect-yvan Architect
```

Validación y comportamiento: Repetido aquí para el catálogo transversal.

Errores comunes: `HUMAN_REVIEW_REQUIRED, HUMAN_AUTHORITY_REQUIRED`

/EXIT

Sintaxis: `/EXIT`

Ejemplo:

```
/EXIT
```

Validación y comportamiento: Finaliza la sesión del intérprete.

Errores comunes: `INVALID_STATE`

3. Tabla de errores

Código	Significado	Cómo resolver
`ACTOR_NOT_FOUND`	El actor no existe. Verifique `/ACTOR LIST`.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`ACTOR_REQUIRED`	Falta actor. Proporcione un actor válido.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`CANDIDATE_NOT_FOUND`	La alternativa/candidato no existe. Use `/ALTERNATIVE LIST`.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`CONFLICT`	El identificador ya existe o hay conflicto de estado.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`CYCLE_NOT_FOUND`	El ciclo no existe. Use `/CYCLE LIST`.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`DECISIONAL_ACTOR_REQUIRED`	El actor no tiene autoridad DECISIONAL.	Consulte la sintaxis del comando y

Código	Significado	Cómo resolver
		verifique el estado, identificadores y autoridad requeridos.
`EVIDENCE_ALREADY_EXISTS`	El evidence_id ya existe; use otro identificador.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`EVIDENCE_NOT_FOUND`	La evidencia no existe.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`EVOLUTION_NOT_FOUND`	La evolución no existe.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`GENERATION_NOT_FOUND`	La generación no existe.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`HUMAN_AUTHORITY_REQUIRED`	Falta autoridad humana explícita.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.

Código	Significado	Cómo resolver
`HUMAN_REVIEW_REQUIRED`	Debe existir HumanReview APPROVED antes de decidir.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`IMPORT_FAILED`	Falló la importación; revise archivos y columnas.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`INTERPRETATION_NOT_FOUND`	La interpretación no existe.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`INVALID_ARGUMENT`	Cantidad u orden de argumentos inválido.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`INVALID_DISTRIBUTION`	Distribución Monte Carlo inválida.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`INVALID_EVIDENCE_TYPE`	Tipo Evidence no permitido.	Consulte la sintaxis del comando y verifique el estado, identificadores y

Código	Significado	Cómo resolver
		autoridad requeridos.
`INVALID_INPUTS`	Parámetros inválidos o fuera de rango.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`INVALID_SCOPE`	Scope espacial o temporal inválido.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`INVALID_SCOPE_RELATION`	Relación multiescala inválida.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`INVALID_STATE`	La operación no es válida en el estado actual.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`MEMORY_NOT_FOUND`	La memoria no existe.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`MEMORY_REVOKED`	La memoria fue revocada y no puede	Consulte la sintaxis del comando y

Código	Significado	Cómo resolver
	aplicarse.	verifique el estado, identificadores y autoridad requeridos.
`METHOD_NOT_FOUND`	Método de simulación o generación no registrado.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`METHOD_TYPE_MISMATCH`	El método no corresponde al tipo de simulación.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`LLM_NOT_CONFIGURED`	LLM no configurado; use método determinista u configure el proveedor.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`OBJECTIVE_DIRECTION_REQUIRED`	Debe indicar MAXIMIZE o MINIMIZE.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`OBJECTIVE_NOT_FOUND`	El objetivo no existe.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.

Código	Significado	Cómo resolver
`PARAMETER_NOT_FOUND`	El parámetro requerido no existe en la alternativa.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`PLANNING_INSTRUMENT_NOT_FOUND`	El instrumento de planificación no existe.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`PRINCIPLE_NOT_FOUND`	El principio de diseño no existe.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`PROJECT_ALREADY_EXISTS`	El project_id ya existe.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`PROJECT_NOT_FOUND`	No existe el proyecto indicado.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`REGULATION_NOT_FOUND`	La regulación no existe.	Consulte la sintaxis del comando y verifique el estado, identificadores y

Código	Significado	Cómo resolver
		autoridad requeridos.
`SCALE_RELATION_REQUIRED`	Falta relación multiescala válida.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`SCENARIO_NOT_FOUND`	El escenario no existe.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`SIMULATION_NOT_FOUND`	La simulación no existe.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`SNAPSHOT_NOT_FOUND`	El snapshot no existe.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`MULTIOBJECTIVE_NOT_FOUND`	El resultado multiobjetivo no existe.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.
`UNKNOWN_COMMAND`	Comando no registrado; consulte `/HELP`.	Consulte la sintaxis del comando y

Código	Significado	Cómo resolver
		verifique el estado, identificadores y autoridad requeridos.
`VETO_BLOCKED`	Un actor con veto bloqueó la operación.	Consulte la sintaxis del comando y verifique el estado, identificadores y autoridad requeridos.

4. Formatos especiales

Timestamp UTC

```
2026-09-16T07:00:00Z
```

JSON inline

```
'{"key":"value"}'
```

Distribuciones Monte Carlo

- `NORMAL`: `{"type":"NORMAL","parameters":{"mean":10,"std_dev":1}}`
- `UNIFORM`: `{"type":"UNIFORM","parameters":{"low":0,"high":100}}`
- `TRIANGULAR`: `{"type":"TRIANGULAR","parameters":{"low":0,"mode":50,"high":100}}`
- Iterations: entero entre 1 y 10000; la semilla CLI actual es `20260916`.

Scopes espaciales

`pais`, `macro_region`, `region`, `provincia_metropoli`, `distrito_ciudad`, `zona_barrio_sector`, `parcela_sitio`, `edificacion`, `sistema`, `espacio`, `objeto`.

Scopes temporales

`proyecto`, `corto_plazo`, `mediano_plazo`, `largo_plazo`, `escenario_2030`, `escenario_2040`, `escenario_2050`.

Evidence types

`DOCUMENT`, `OBSERVATION`, `MEASUREMENT`, `REFERENCE`, `TESTIMONY`, `NORMATIVE`, `OTHER`.

5. Cobertura y límites

Esta referencia incluye 100 fichas de comandos o superficies relacionadas. El catálogo efectivo del dispatcher se obtiene siempre con `/HELP` y corresponde a los comandos implementados en `src/sicl/cli.py`.

Source y Council están documentados como HTTP-only. `/REPORT`, `/TRADEOFFS` y `/DASHBOARD` se conservan como nombres históricos o conceptuales cuando no son comandos independientes del dispatcher actual; no deben presentarse como comandos CLI ejecutables.

Invariante constitucional: una Recommendation, simulación, evaluación, comparación o salida de agente nunca crea una Decision automáticamente. La Decision requiere HumanReview y autoridad humana explícitas.

Estado vigente de capacidades — actualización 2026-09-16

> Esta sección supersede las afirmaciones históricas de este documento que indiquen que GIS, BIM o el Copiloto local están fuera de alcance. El baseline vigente incluye RFC-027, RFC-028 y RFC-029.

El Core actual incluye importación IFC física de solo lectura, snapshots BIM persistentes, BIMChangeSet en modo PREVIEW, detección de conflictos BIM, contratos de exportación que exigen HumanReview, persistencia SQLite de ParcelSnapshot, catálogo GIS por jurisdicción, caducidad y ParcelBoundaryConflict, consulta OGC API Features y traducción Ollama a comandos SICL en modo PREVIEW_ONLY.

Estas capacidades no otorgan autoridad automática. El Copiloto no ejecuta comandos ni crea decisiones. Los adaptadores Revit y Archicad requieren sus hosts y SDKs nativos. La exportación BIM queda pendiente de ejecución externa después de una HumanReview aprobada. Las fuentes catastrales requieren registro explícito, licencia y validación humana.

La referencia operativa es `PRODUCTION_CORE_DEPLOYMENT_MANUAL.md`. La suite E2E se ejecuta con `PYTHONPATH=.:src pytest -q tests/test_e2e_ollama_sicl.py -ra`.